

Chapter 5* Shading Practice

• 核心

• 光栅化成像渲染流水线

- 步骤一：模型导入（obj文件，内部包含构成模型的所有平面）
- 步骤二：光栅化程序初始化（载入着色器，初始化Color Buffers和Depth Buffers，载入mvp变换矩阵）
- 步骤三：开始主渲染流水线
 - 步骤3.1：对于构成模型的每个三角形，依次对其应用mvp变换矩阵、Perspective Division、Viewport transformation变换矩阵，将最后得到的三角形送入rasterizer
当然为了在rasterizer中准确地计算着色，只应用mv矩阵得到的viewspace坐标也会被保留
 - 步骤3.2：光栅化
 - 步骤3.2.1：确认三角形的**bounding box**（优化复杂度）
 - 步骤3.2.2：对于bounding box内的每个像素，确认其是否在三角形内，若是则继续进行以下色彩确定流程（可以采用**Super Sampling**形成色彩有无的线性过渡）
 - 步骤3.2.3：对于三角形内的每个像素，**插值**计算其各种属性（包括但不限于深度、色彩、法线、纹理坐标）
 - 步骤3.2.4：根据**深度测试**的结果判断是否对该像素着色（即当前像素是否被别的像素覆盖）
 - 步骤3.2.5：着色器着色（如phong shader, texture shader, bumpmapping shader, normalmapping shader, displacementmapping shader等）
该作业只考虑了fragment shader, vertex shader的作用暂不明确

• 其他记录

• 其它

• Eigen

• 定义

- 矩阵赋值前需要先通过 `Eigen::Matrix::f::Identity()` 初始化

• 运算

- 了解 `cwiseProduct` 的用法：将矩阵/向量对应分量相乘得到新的矩阵/向量
`a.cwiseProduct(b)`
- Scalar类型的变量有自动类型转换的特性，但是对于不明确类型无法转换，从而导致报错。对此的解决方法是明确类型（如取 `max` 时 `max(0.f, scalar)`）或强制类型转换Scalar类型的变量

• 规范化

- `norm()` : 对于向量计算其二范数
- `normalize()` : 对于向量将其本身规范化
- `normalized()` : 对于向量将其规范化后得到新向量